

Activity Based Surveillance Video Segmentation

Tao Xiang and Shaogang Gong

Department of Computer Science

Queen Mary, University of London, London E1 4NS, UK

1. Introduction

There is an urgent need for applications that allow for intelligent and quick access to the vast amount of surveillance video footages generated everyday. In this paper, we address the issue of decomposing surveillance video sequences into segments each of which contains frames of similar content. Most previous temporal video segmentation approaches are based on shot change detection. Since shots do not exist in surveillance videos, we propose to represent video content at the semantic level and analyse the video structure based on ‘what is happening’ rather than ‘what is present’ in the scene. In particular, we consider developing a suitable representation of video which captures the spatio-temporal characteristics of events occurred in the scene over time. A scene vector is constructed as a cumulative histogram of all the event classes in a multivariate feature space where each dimension corresponds to one particular event class. Given this representation, a novel on-line segmentation algorithm is proposed to detect significant video content changes and segment continuous videos into activities. Each activity is composed of events of different classes whose occurrences follow certain temporal order.

2. Video Representation and Segmentation



Figure 1. Event detection with different classes of events highlighted using bound-boxes in different colours.

We adopt an approach for automatic event detection and classification without object-centred tracking [1]. The number of event classes captured in a video is determined by automatic model order selection. An example of event detection and classification for an aircraft docking video (we refer this video as video 1 henceforth) is shown in Figure 1. Eight classes of events are detected automatically, each of which corresponds to different movement patterns by different objects. Given event classification over the training videos, a scene vector is constructed for each frame of the same videos (or of novel videos captured at the same scene). Let us first consider a scene vector $\mathbf{sv}_i$ for a video frame i is defined as:

$$\mathbf{sv}_i = [SV_i^1, SV_i^2, \dots, SV_i^K] \quad (1)$$

where K is the number of event classes detected over the training video (e.g. $K = 8$ for video 1). The value of the k^{th} element of $\mathbf{sv}_i$, SV_i^k is the number of events of the k^{th} event class detected in frame i . However, directly using the scene vector for video segmentation can cause problems due to the discrete nature of $\mathbf{sv}_i$. To overcome this problem, let us now reconsider the scene vector being defined as a cumulative histogram of $\mathbf{sv}_i$ up to frame t . The k^{th} element of the new scene vector (denoted as $\widetilde{\mathbf{sv}}_t$) is computed as:

$$\widetilde{SV}_t^k = \sum_{i=1}^t SV_i^k \quad (2)$$

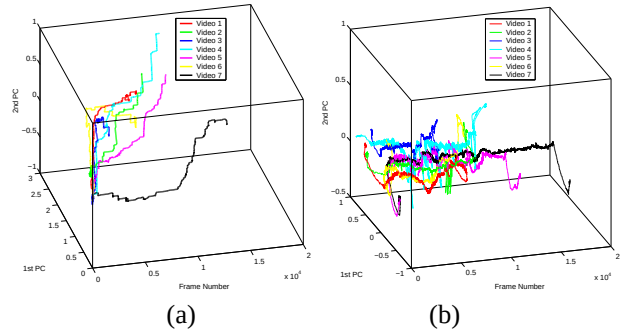


Figure 2. (a) and (b) are the first 2 principal components of $\widetilde{\mathbf{sv}}_t$ and colour histogram for seven aircraft docking videos respectively.

Figure 2 (a) shows the evolution of the first two principal components of $\widetilde{\mathbf{sv}}_t$ from seven different sequences of aircraft docking lasting from 6470 to 17262 frames per sequence. Among them, video 1 to 6 follow the typical video structure with cargo services performed before catering services while video 7 has different video structure with cargo services performed after catering services. It can be seen that the video trajectory of video 7 are distinctively different from those of the other 6 videos. For comparison, the same video sequences are also represented using colour histogram. We define 4 histogram bins for each of the three colour channels of the RGB colour space. Separate RGB colour histograms are concatenated into one with 12 components. Figure 2 (b) illustrates the colour histogram representation of the same seven videos. It is evident from Figure 2 (b) that the colour histogram video trajectories are less smooth compared to the scene vector video trajectories. It is also noted that the trajectory corresponding to video 7 is similar to other trajectories under this colour histogram representation.

After mapping a video sequence into a cumulative scene

vector histogram trajectory¹, the breakpoints on the trajectory correspond to the video content change points. The video segmentation problem then becomes a trajectory breakpoint detection problem. One of the most commonly used approaches for trajectory segmentation is Piecewise Linear Approximation (PLA) which refers to the approximation of a trajectory of length T with K straight lines by either Linear Interpolation or Linear Regression. In this paper, a novel on-line algorithm is proposed based on the Forward-Backward Relevance (FBR) principle, as follows:

1. Set the starting point of the first segment at the first data point of the trajectory.
2. After the minimal segment length L_{min} ;
3. compute the relevance $R(currentVertex) = R(startingPoint, currentVertex, currentVertex + L_{min})$ for the current vertex;
4. **if** $R(currentVertex)$ is smaller than a threshold Th , move to the next vertex and **goto** 3;
5. **else** the current vertex is a breakpoint; set the starting point of the next segment at the current vertex and **goto** 2.

The relevance measure R for a vertex v on the trajectory depends on v and its two neighbour vertices u and w :

$$R(u, v, w) = d(v, u) + d(v, w) - d(u, w) \quad (3)$$

where function $d(\cdot)$ can be any distance or similarity measures such as Euclidean distance. Compared to Linear Interpolation or Linear Regression, the computation of R is much faster especially when the dimensionality of the trajectory is high. Two parameters, L_{min} and Th need to be determined. We propose to perform an off-line segmentation algorithm Discrete Curve Evolution (DCE) [3] on the training data. L_{min} is then set as half of the length of the shortest segment in the training data and Th is computed as the average value of $R(currentVertex)$ of the breakpoints detected by DCE. One important advantage of our algorithm is that for each new data point, we compute $R(currentVertex)$ which involves only computing 3 distances. The computation complexity is independent of the segment length and thus much lower than the classic on-line Sliding Window algorithm and its variations [2].

3. Experiments

We compare the performance of the off-line Discrete Curve Evolution (DCE) and the on-line Forward-Backward Relevance (FBR) algorithms on 7 different aircraft docking videos represented by scene vector $\tilde{s}\mathbf{v}_t$ (see Figure 2 (a)). The videos are manually segmented into activities to give the ground truth of the breakpoints. The number of segments (activities) in each sequence was between 6 to 11 with the length of these segments ranging from 196 to 4350 frames. Three videos were used as training data and the others as test data. This was repeated for four times with different videos as the training and test data. The performance of segmentation was measured by the correct detection rate and false positives of breakpoint detection. When a breakpoint is detected within 100 frames of the true breakpoint, it is regarded as being correctly detected. Altogether,

¹More precisely, it is a video polyline due to the discrete nature of the image frames.

there were 135 true breakpoints in the test datasets. Table 1 shows that the FBR algorithm gave slightly worse results compared to the DCE algorithm. Figure 3 shows an example FBR breakpoint detection result. The bottom sub-figure shows video 2 represented by the scene vector $\tilde{s}\mathbf{v}_t$ with the 7 detected breakpoints overlapped on the video trajectory (the trajectory evolves from left to right). The top two rows of Figure 3 are the frames corresponding to the detected breakpoints. We also performed DCE and FBR on the colour histogram video representation following the same experimental procedure. From Table 2, it is clear that both DCE and FBR failed to produce any meaningful segmentation on the same 7 aircraft docking videos.

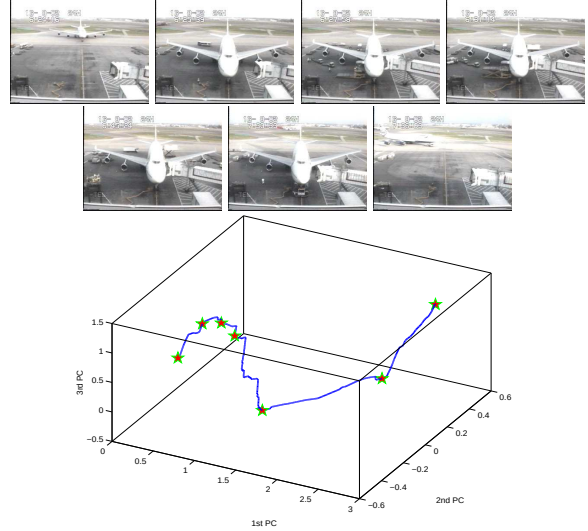


Figure 3. An example breakpoint detection result by FBR algorithm.

	DCE	FBR
Correct Detection Rate	77.8 %	72.6%
False Positives	39	64

Table 1. Scene vector trajectory segmentation.

	DCE	FBR
Correct Detection Rate	30.4%	28.9%
False Positives	103	89

Table 2. Colour histogram trajectory segmentation.

References

- [1] S. Gong and T. Xiang. Recognition of group activities using dynamic probabilistic networks. In *ICCV*, 2003.
- [2] E. Keogh. An online algorithm for segmenting time series. In *ICDE*, pages 289–296, 2001.
- [3] L. Latecki and R. Lakamper. Convexity rule for shape decomposition based on discrete contour evolution. *CVIU*, 73:441–454, 1999.